Dataset Guide

The following section details the data and an example of usage towards building privacy profiles is given at the end of this page. Please contact us if you find any discrepancy from the documentation to the actual data.

Please cite the following article if you use our dataset in your research: Mendes, R., Brandão, A., Vilela, J. P., and Beresford, A. R.. Effect of User Expectancy on Mobile App Privacy: A Field Study. In 2022 IEEE international conference on pervasive computing and communications (PerCom).

The Data

The data files are in JSON format and each file is an array of objects with keys and values as specified below. Please note that some of the data might not be present at times as it was not collected. The indication of `nullable` in the specification indicates a field that might be missing. For instance, the `contextData` is missing when the `answerType` in `permissionDialogData` is either `CACHE_ANSWERED` or `UNHANDLED`, as we only collect the context when there is a permission prompt.

An example of a data file is given at the end of this section.

- **userID**: string. Unique user identifier, formatted as `<campaign_id>.<phone_id>`.
- **permissionDialogData**: object. Contains the data pertaining to a permission dialog.
  - **permissionDialogDataId**: long. Internal identifier of the permission dialog data.
  - **requestingApplicationInfo**: object. Contains data about the requesting application.
    - **packageName**: string. The *hashed* package name of the app.
    - **flags**: nullable integer. Application flags as given by ApplicationInfo flags.
    - **isSystemApp**: nullable boolean. Whether the app is a system app or a third-party app.
    - **isForeground**: nullable boolean. Whether the app is currently in foreground or background.
    - **category**: string. The category of this app in the play store. This field was filled in a post-process through a python google play scrapper: google-play-scraper.
  - **checkedPermission**: string. The name of the permission being requested as declared in the manifest. Default android permissions can be found here.
  - **checkedPermissionGroup**: nullable string. The group of the `checkedPermission` if any. Default android permission groups can be found here
  - **timestamp**: long. The epoch time of this permission request.
  - **answerType**: string. The value can be one of: `USER_ANSWERED`, `CACHE_ANSWERED`, `TIMEDOUT`, `DISMISSED`, `UNHANDLED`. Requests with `UNHANDLED` correspond to requests that CM-NPM failed to handle due to potential communication errors.
  - **grantResult**: integer. Whether the permission was granted or denied by the user. The value is either `PackageManager.PERMISSION_GRANTED` (0) if the permission is granted, `PackageManager.PERMISSION_DENIED` (-1) if the permission is denied, or Integer.MIN_VALUE (-2147483648) if there was no answer.
  - **selectedSemanticLoc**: nullable string. The user semantic location given as input by the participant. The value can be one of: `Home`, `Work`, `Home`, `Travelling` and `Other`.
  - **wasRequestExpected**: nullable integer. Participant input on whether the request was expected. Value is 2 if it was expected, 1 if it was unexpected or 0 if the participant was not sure.
- **contextData:** nullable object. Contains the data related to the participant and device context at the time of the permission dialog. This field does not exist when **answerType** in **permissionDialogData** is `CACHED_ANSWERED` or `UNHANDLED`.
  - **contextDataId**: long. Internal identifier of the context data object.
  - **bootTime**: long. The time in milliseconds of the last boot. **Note:** campaign 3 is missing this field.
  - **permissionDialogDataOwnerId**: long. The id of the **permissionDialogData** object that this contextData pertains to.
  - **topRunningApplicationInfo**: nullable object. The application that is currently being shown to the user as top application. Contains the same fields as **requestingApplicationInfo**.
  - **isTopAppRequestingApp**: nullable bool. Whether the requesting app is the top foreground app -- app that is currently being used by the user.
  - **foregroundRunningApplicationInfoArray**: nullable array of objects. An array of the applications that are visible to the user at the time of the permission dialog. Each application has the same fields as **requestingApplicationInfo**.
  - **backgroundRunningApplicationInfoArray**: nullable array of objects. An array of the applications that are running in the background at the time of the permission dialog. Each application has the same fields as **requestingApplicationInfo**.
  - **networkStatus**: string. The network status at the time of the permission dialog. The value is one of: `DISCONNECTED`, `NOT_METERED` or `METERED`.
  - **screenIsInteractive**: boolean. Whether the screen is interactive (`true`) or not (`false`) as given by the PowerManager isInteractive().
  - **isKeyguardLocked**: boolean. Whether the keyguard is locked (`true`) or unlocked (`false`) as given by the KeyguardManager isKeyguardLocked().

- **dockState**: integer. The dock state at the time of the dialog. It is the integer value obtained from EXTRA_DOCK_STATE.
- **callState**: integer. The current call state as returned by the TelephonyManager getCallState().
- **plugState**: integer. The current plug (charging) state as returned from the field EXTRA_PLUGGED. Value is 0 if it is not plugged, 1 if it is plugged to an AC charger, 2 if it is plugged to an USB port and 3 if the power source is wireless.
- **isInEvent**: nullable boolean. Is 1 if the user is in an event at the current time according to his calendar and 0 otherwise. Field might be missing as some users have not synced their calendars.

Example File


```
[
    {
        "userID": "1.5eb63bbbe01eeed093cb22bb8f5acdc3",
        "contextData": {
            "backgroundRunningApplicationInfoArray": [
                {
                    "packageName": "0800fc577294c34e0b28ad2839435945",
                    "flags": 550027079,
                    "isForeground": true,
                    "isSystemApp": false,
                    "category": "WEATHER"
                }
            ],
            "isInEvent": 1,
            "callState": 0,
            "contextDataId": 1,
            "dockState": 0,
            "foregroundRunningApplicationInfoArray": [
                {
                    "packageName": "58833651db311ba4bc11cb26b1900b0f",
                    "flags": 550027079,
                    "isForeground": true,
                    "isSystemApp": true,
                    "category": "GAME"
                }
            ],
            "isKeyguardLocked": false,
            "networkStatus": "NOT_METERED",
            "permissionDialogDataOwnerId": 1,
            "plugState": 0,
            "screenIsInteractive": true,
            "isTopAppRequestingApp": 1,
            "topRunningApplicationInfo": {
                "flags": 550027079,
                "isForeground": true,
                "isSystemApp": true,
                "packageName": "58833651db311ba4bc11cb26b1900b0f",
                "category": "GAME"
            }
        },
        "permissionDialogData": {
            "answerType": "USER_ANSWERED",
            "checkedPermission": "android.permission.WRITE_EXTERNAL_STORAGE",
            "checkedPermissionGroup": "android.permission-group.STORAGE",
            "grantResult": 0,
            "permissionDialogDataId": 1,
            "requestingApplicationInfo": {
                "flags": 550027079,
                "isSystemApp": true,
                "packageName": "58833651db311ba4bc11cb26b1900b0f",
                "category": "GAME"
            },
            "selectedSemanticLoc": "Home",
            "timestamp": 123123123,
            "wasRequestExpected": 1
        }
    }
]
```

Building Profiles

What follows in an example on using the dataset to build privacy profiles. The example is in the form of a jupyter notebook, so the code appears in gray and the outputs right below the code. This code can be found in https://github.com/rscmendes/cop-mode-profiles.

Initial Setup

In [93]:

```
import getpass
import os
import warnings
warnings.filterwarnings("ignore")
from datetime import datetime
from pathlib import Path
from urllib.parse import quote_plus
import math
import sys


import matplotlib.pyplot as plt
plt.rc('axes', axisbelow=True)  # sets grid under plot
plt.rcParams.update({'font.size': 16})
import matplotlib.cm as cm
import numpy as np
import pandas as pd
from dotenv import load_dotenv
from pymongo import MongoClient
import pickle

from sklearn.feature_selection import chi2
from sklearn.metrics import confusion_matrix, f1_score, plot_roc_curve, accuracy_score, precision_recall_fsc
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split, cross_validate, KFold
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import LinearSVC, SVC
from sklearn.decomposition import FactorAnalysis
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans, MiniBatchKMeans, DBSCAN
# fill missing data
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# warnings.filterwarnings("ignore")

base_imgs_path = Path().resolve()\
    .parent\
    .joinpath("outputs")

grant_result_imgs_path = base_imgs_path\
    .joinpath('grantResult')\
    .resolve()

predictions_path = grant_result_imgs_path.joinpath('prediction').resolve()

profiles_imgs_path = grant_result_imgs_path.joinpath('profiles').resolve()

saves_path =  grant_result_imgs_path.joinpath('saves').resolve()

dotenv_path = Path().resolve().joinpath('.env')
load_dotenv(dotenv_path)
mongo_user = os.environ.get("MONGO_USER")
mongo_password = os.environ.get("MONGO_PWD")
mongo_host = os.environ.get("MONGO_HOST")

if not mongo_user or not mongo_password:
    print("Mongo user not found in MONGO_USER and MONGO_PWD environment variables...")
    mongo_user = input('Insert mongo user: ')
    mongo_password = getpass.getpass('Insert mongo user password: ')

if not mongo_host:
    mongo_host = input("Insert mongo host: [localhost:27017] ") or "localhost:27017"

mongo_uri = "mongodb://%s:%s@%s" % (quote_plus(mongo_user), quote_plus(mongo_password), mongo_host)

# print(mongo_uri)
client = MongoClient(mongo_uri)
db = client.cop_mode_export

def clean_permission_feature_names(feature):
    if "permission" in feature:
        return "permission_" + feature.split('.')[-1]
    return feature

def _get_original_data(db,
                       drop_features,
                       verbose=True):
    # removed 'isKeyguardLocked' and 'screenIsInteractive' because CM-NPM now only asks if screen is interac
    # removed dockState because we have no record where it is different than 0
    # removed checkPermission because Android's permission manager operates at the group level
    original_data = pd.DataFrame(db.data.aggregate([
        {"$match": {"permissionDialogData.answerType": "USER_ANSWERED"}},
        {"$project": {"_id": 0,
                      "timestamp": "$permissionDialogData.timestamp",
                      "callState": "$contextData.callState",
```

```
                    # "dockState": "$contextData.dockState",
                    # "isKeyguardLocked": "$contextData.isKeyguardLocked",
                    # "screenIsInteractive": "$contextData.screenIsInteractive",
                    "networkStatus": "$contextData.networkStatus",
                    "plugState": "$contextData.plugState",
                    # "checkedPermission": "$permissionDialogData.checkedPermission",
                    "checkedPermissionGroup": "$permissionDialogData.checkedPermissionGroup",
                    "grantResult": "$permissionDialogData.grantResult",
                    "selectedSemanticLoc": "$permissionDialogData.selectedSemanticLoc",
                    "wasRequestExpected": "$permissionDialogData.wasRequestExpected",
                    "category": "$permissionDialogData.requestingApplicationInfo.category",
                    "userID": 1,
                    "foregroundRunningApplicationInfoArray": "$contextData.foregroundRunningApplicationInf
                    "topRunningApplicationPackageName": "$contextData.topRunningApplicationInfo.packageNam
                    "requestingApplicationPackageName": "$permissionDialogData.requestingApplicationInfo.p
                    "isInEvent": "$contextData.isInEvent",
                    "isTopAppRequestingApp": "$contextData.isTopAppRequestingApp",
                    "isForeground": "$permissionDialogData.requestingApplicationInfo.isForeground",
                    }
        }
    ]))
    # display(len(original_data))

    if verbose:
        nr_running_apps_missing_docs = db.data.find(
            {"permissionDialogData.answerType": "USER_ANSWERED",
             "$or": [{"contextData.foregroundRunningApplicationInfoArray": None},
                     {"contextData.topRunningApplicationInfo": None}]},
            {"permissionDialogData": 1,
             "contextData.foregroundRunningApplicationInfoArray": 1,
             "contextData.topRunningApplicationInfo.packageName": 1}).count()

        print(f"We are missing information about the running and top apps on {nr_running_apps_missing_docs}

    original_data.drop(["foregroundRunningApplicationInfoArray",
                        "topRunningApplicationPackageName"], axis=1, inplace=True)

    datetime_col = [datetime.fromtimestamp(x / 1000) for x in
                    original_data["timestamp"]]
    original_data["hour"] = [date.hour for date in datetime_col]
    original_data["isWeekend"] = [(date.weekday() >= 5) for date in
                                  datetime_col]
    # display(original_data)

    if verbose:
        nr_requests_in_event = original_data['isInEvent'].loc[original_data['isInEvent'] == 1].count()
        print(f"Requests where the user is in event: "
              f"{nr_requests_in_event} ({round(nr_requests_in_event / len(original_data) * 100, 2)}%)")

    original_data = original_data.loc[:,
                    [column for column in original_data.columns
                     if column not in drop_features]]
    if verbose:
        print("Full dataset features: \n\t - {}.".format('.\n\t - '.join(original_data.columns)))

    return original_data


def _get_encoded_scaled_x_y(original_data, y_feature, verbose=True):
    X_one_hot_enc = pd.get_dummies(original_data)
    # print(X_one_hot_enc.columns)

    nr_requests_before_dropping_na = len(X_one_hot_enc)
    X_one_hot_enc.dropna(inplace=True)  # some isForeground are nan

    if (verbose):
        print(
            f"\nDropped {nr_requests_before_dropping_na - len(X_one_hot_enc)} requests due to nan features")
        if y_feature == 'wasRequestExpected':
            was_request_expected_df = X_one_hot_enc[
                'wasRequestExpected'].replace(
                {0: "UNKNOWN", 1: "UNEXPECTED", 2: "EXPECTED"}).value_counts()
            was_request_expected_df = pd.DataFrame({
                'count': was_request_expected_df,
                'relative_count': round(
                    was_request_expected_df / was_request_expected_df.sum() * 100,
                    2)
            })
            print(was_request_expected_df)

    if y_feature == 'wasRequestExpected':
        nr_requests_before_dropping_unknown = len(X_one_hot_enc)
        X_one_hot_enc = X_one_hot_enc.loc[X_one_hot_enc['wasRequestExpected'] != 0]

        if verbose:
            print(
                f"Dropped {nr_requests_before_dropping_unknown - len(X_one_hot_enc)} requests where wasReque
            print(
                f"Remaining requests: {len(X_one_hot_enc)} ({round(len(X_one_hot_enc) / nr_requests_before_d

    scaler = MinMaxScaler()
    X_one_hot_enc = pd.DataFrame(scaler.fit_transform(X_one_hot_enc),
                                 columns=X_one_hot_enc.columns)
```

```
    X_col_idx = (X_one_hot_enc.columns != y_feature)
    if y_feature == "wasRequestExpected":
        # we wouldn't have the grant result when predicting expectancy
        X_col_idx = np.logical_and(X_col_idx, (X_one_hot_enc.columns != "grantResult"))

    X, y = X_one_hot_enc.loc[:, X_col_idx], X_one_hot_enc[y_feature]

    if verbose:
        print("Features: \n\t - {}.".format('.\n\t - '.join(X.columns)))
    return X, y
```

COP-MODE

COP-MODE is a research project funded through the European Union's Horizon 2020 research and innovation programme under the NGI_TRUST grant agreement no 825618

Contact

cop-mode@dei.uc.pt

Dataset Characterization

Number of users

In [33]:
```
len(db.data.distinc
```

Out[33]:
```
93
```

Total number of requ

In [34]:
```
nr_total_requests = db.data.count_doc
print(nr_total_requests)
```

```
2180302
```

Distribution of answer type

In [38]:
```
data = pd.DataFrame(db.data.aggregate([
    # { "$match": { "permissionDialogData.answerType": "USER_ANSWERED" } },
    { "$group": { "_id": "$permissionDialogData.answerType", "count": {"$sum": 1}} },
    { "$sort": {"count": 1}}
]))
display(data)
```

© 2019-2020 COP-MODE

	_id	count
0	DISMISSED	1108
1	USER_ANSWERED	65261
2	TIMEDOUT	74116
3	UNHANDLED	96834
4	CACHE_ANSWERED	1942983

Answers per user

In [36]:
```
data = pd.DataFrame(db.data.aggregate([
    { "$match": { "permissionDialogData.answerType": "USER_ANSWERED" } },
    { "$group": { "_id": "$userID", "count": {"$sum": 1}} },
    { "$sort": {"count": 1}}
]))
# with pd.option_context('display.max_rows', None):
#     print(data)
# data.boxplot(column='count')
print(data['count'].describe())
```

```
count      93.000000
mean      701.731183
std      1096.330115
min        51.000000
25%       270.000000
50%       446.000000
75%       680.000000
max      8358.000000
Name: count, dtype: float64
```

Privacy Profiles

Privacy Profiles using the category-permission pair

This methodology follows the approach described in Liu, B., Andersen, M. S., Schaub, F., Almuhimedi, H., Zhang, S. A., Sadeh, N., ... & Acquisti, A. (2016). Follow my recommendations: A personalized privacy assistant for mobile app permissions. In Twelfth Symposium on Usable Privacy and Security (SOUPS 2016) (pp. 27-41).

Load the data

In [129…
```
data = _get_original_data(db,
                          drop_features=("timestamp",
```

```
                                        "requestingApplicationPackageName"),
                            verbose=True)

# Put all games in same category
data['category'] = data['category'].apply(lambda x: x.split('_')[0] if x.startswith('GAME_') else x)

# Clean permission names
data['checkedPermissionGroup'] = data['checkedPermissionGroup'].map(lambda x: x.split('.')[-1])
data = data.rename(columns={'checkedPermissionGroup': 'permission'})
data = data.loc[data['permission'] != 'SENSORS']  # remove SENSORS permissions -- only 1 permission
```

```
We are missing information about the running and top apps on 4448 records
Requests where the user is in event: 7655 (11.73%)
Full dataset features:
        - userID.
        - callState.
        - networkStatus.
        - plugState.
        - checkedPermissionGroup.
        - grantResult.
        - selectedSemanticLoc.
        - wasRequestExpected.
        - category.
        - isTopAppRequestingApp.
        - isForeground.
        - isInEvent.
        - hour.
        - isWeekend.
```

Form the tensor data

Each user becomes a row where each column is a pair category-permission, whose value is the average grant result of the user (value between -1 and 0) for that specific pair.

Missing values correspond to cases where the user has no answered permission for the given pair. Data must be inputted in order to form the profiles using hierarchical clustering. We use SciKit's IterativeImputer.

In [130…

```
def _get_tensor_data(data, clustering_features, y_feature, random_state):
    clustering_features_unique_tuples = data[clustering_features].drop_duplicates()

    clustering_df = data.groupby(['userID'] + clustering_features).mean([y_feature])
    clustering_df = clustering_df[[y_feature]]
    # print(clustering_df)

    all_users = data['userID'].unique()
    for u_index, u in enumerate(all_users):
        for i, row in clustering_features_unique_tuples.iterrows():
            try:
                index = tuple([u] + row.to_list())
                clustering_df.loc[index][y_feature]
            except KeyError:
                clustering_df.loc[index, y_feature] = np.nan
    clustering_df = clustering_df.reset_index()

    # print(clustering_df)
    missing_data_count = clustering_df[y_feature].isna().sum()
    all_data_count = len(clustering_df)
    print(f"Percentage of missing data for dataset with feature {clustering_features} is: {round(missing_dat

    imp_mean = IterativeImputer(random_state=random_state)
    imp_full_clustering_dataset_enc = pd.DataFrame(
        # IMP: get_dummies puts all encoded columns at the end, but keeps continuous at start
        # So, we put the y_feature first so that the index is 0
        imp_mean.fit_transform(pd.get_dummies(clustering_df[[y_feature] + clustering_features])))

    y_feature_index = 0
    clustering_df[y_feature] = imp_full_clustering_dataset_enc.loc[:, y_feature_index]
    # print(clustering_df)

    clustering_df = clustering_df.set_index(['userID'] + clustering_features)
    col_names = []
    tensor_data = []
    for u_index, u in enumerate(all_users):
        user_data = []
        # print(data.loc[u])
        for i, row in clustering_features_unique_tuples.iterrows():
            if u_index == 0:
                col_names.append('|'.join(row.astype('str')))

            # no empty data
            index = tuple([u] + row.to_list())
            value = clustering_df.loc[index][0]
            user_data.append(value)
        # print(user_data)
        tensor_data.append(user_data)

    tensor_data = pd.DataFrame(tensor_data, index=all_users, columns=col_names)
    # print(tensor_data)
    return tensor_data

clustering_features = ['category', 'permission']
y_feature = 'grantResult'
```

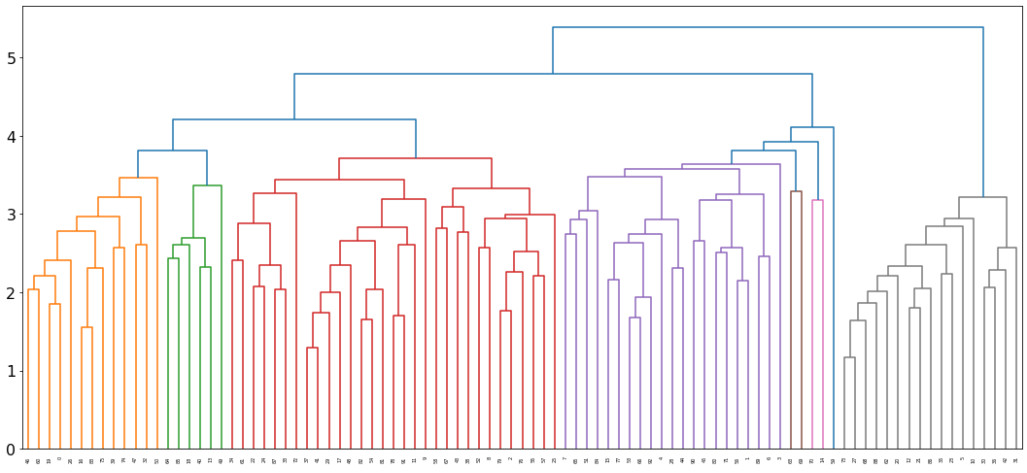
```
y_feature = "grantResult"
random_state = 42
tensor_data = _get_tensor_data(data, clustering_features, y_feature, random_state)
```

```
Percentage of missing data for dataset with feature ['category', 'permission'] is: 79.55.
```

Dendogram

In [131…

```
linked = linkage(tensor_data, "complete", optimal_ordering=True)
plt.figure(figsize=(15, 7))
dendrogram(linked, orientation="top", distance_sort="descending", show_leaf_counts=True)
plt.tight_layout()
plt.show()
```

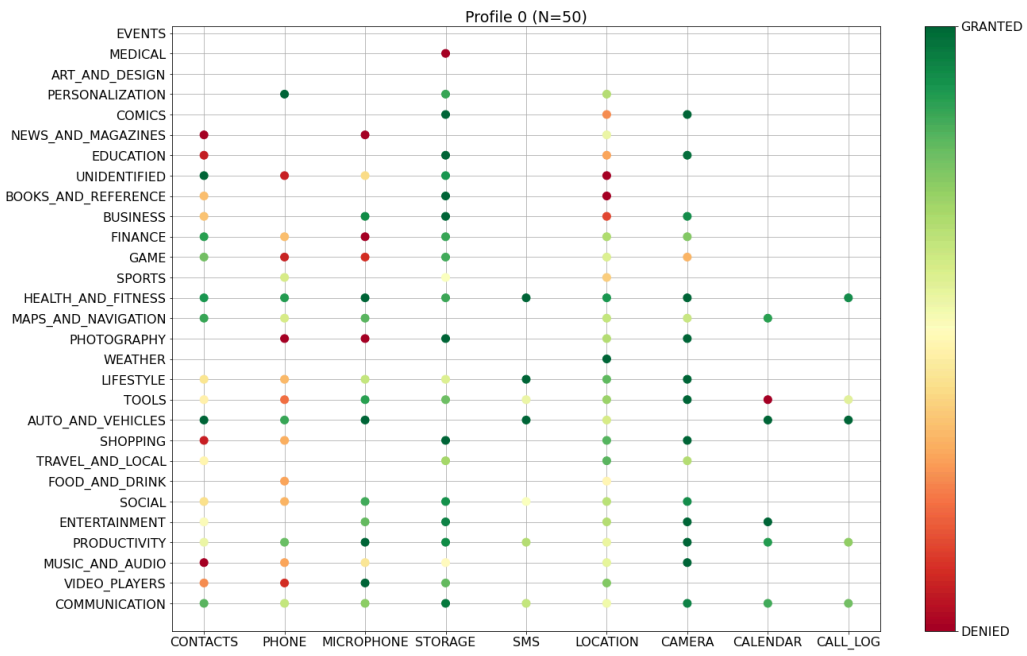


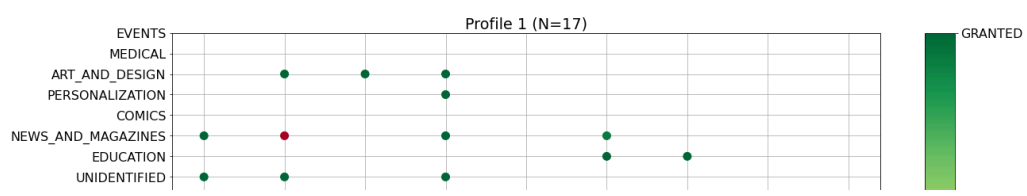
From the dendogram, we can select where to perform the "cut" to have the privacy profiles.

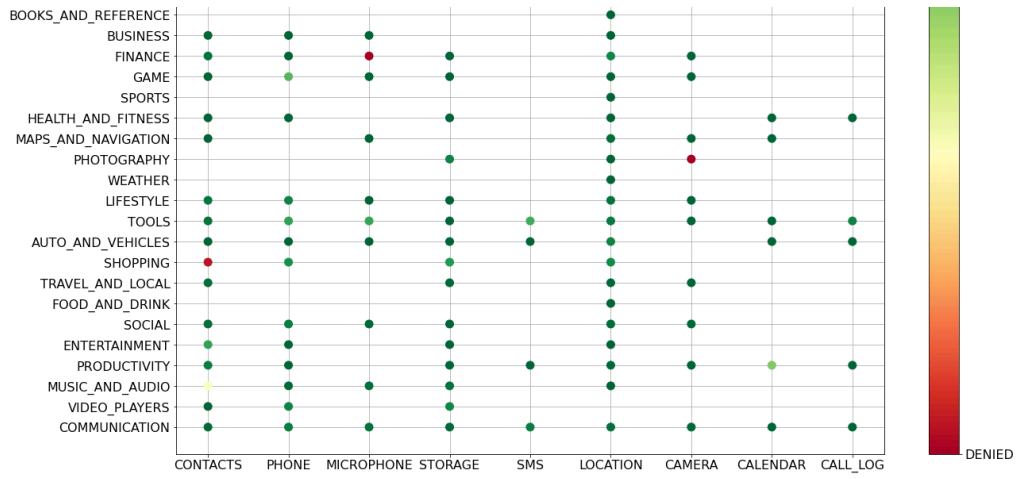
For an illustrative example, we will select 3 profiles and plot them.

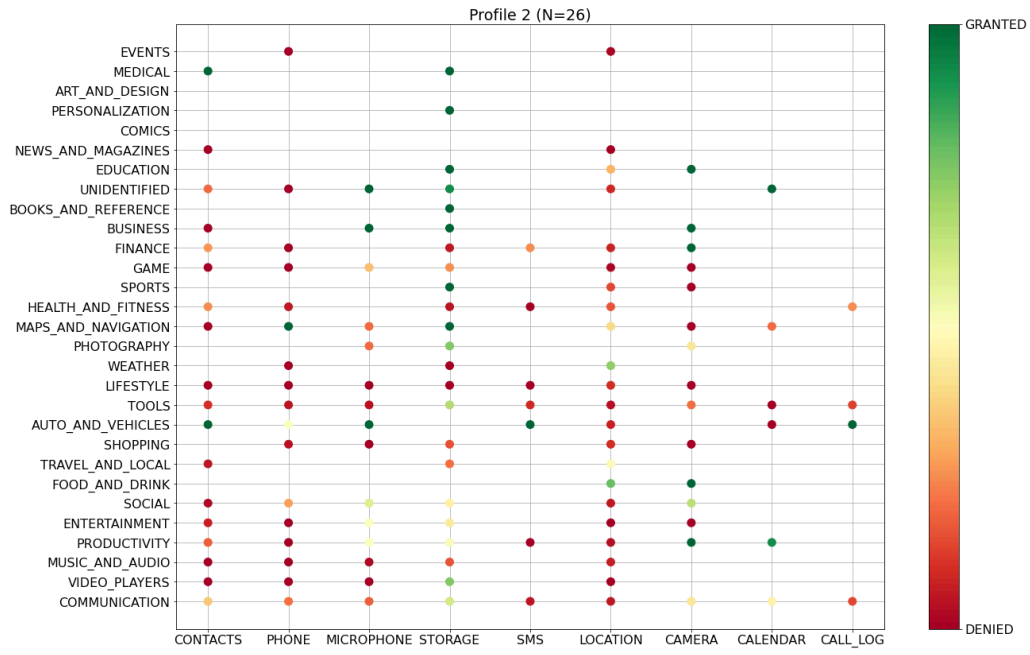
In [132…

```
cluster = AgglomerativeClustering(n_clusters=3, affinity="euclidean", linkage="complete")
cluster.fit_predict(tensor_data)
labels_dict = {userID: label for userID, label in
               zip(tensor_data.index, cluster.labels_)}
data["hc_label"] = [labels_dict[userID] for userID in data["userID"]]
# tensor_dataset["hc_label"] = cluster.labels_
permissions = list(data["permission"].unique())
perm_ticks_map = {tick: i for i, tick in enumerate(permissions)}
categories = list(data["category"].unique())
cat_ticks_map = {tick: i for i, tick in enumerate(categories)}

# cmap = plt.cm.get_cmap("RdYlGn")
for c in list(set(cluster.labels_)):
    plot_data = data.loc[data["hc_label"] == c, :].groupby(clustering_features).mean()
    plot_data.reset_index(inplace=True)
    fig = plt.figure(figsize=(20, 14))  # TODO: fix size
    ax = fig.add_subplot()
    x = [perm_ticks_map[perm] for perm in plot_data["permission"]]
    y = [cat_ticks_map[cat] for cat in plot_data["category"]]
    scatter_plot = ax.scatter(x, y,
        c=plot_data[y_feature], cmap="RdYlGn",
        s=100)  # , s=100
    ax.set_xticks(range(len(permissions)))
    ax.set_xticklabels(permissions)
    ax.set_yticks(range(len(categories)))
    ax.set_yticklabels(categories)

    nr_users_in_c = len(data.loc[data["hc_label"] == c, "userID"].unique())
    ax.set_title(f"Profile {c} (N={nr_users_in_c})")
    ax.grid()
    cbar = fig.colorbar(scatter_plot, ticks=[-1, 0])
    cbar.ax.set_yticklabels(['DENIED', 'GRANTED'])
```









From the plots above, we can see that participants in profile 1 (17 participants) allow most requests, therefore these are typically referred to as "The unconcerned". In contrast, participants in profile 2 (26 participants) mostly deny permission requests. These are the "Privacy Conscious". However, most participants are in profile 0, where there seems to bee a more diverse response. Potentially, this profile could have been further divided, by increasing the number of clusters. Note that increasing the number of clusters degrades the interpretability, but might allow for a better separation of behaviors.

n-Dimensional Privacy Profiles

The previous profiles were built using just the category-permission pair. However, we can use other features and even more features, such as contextual features, to form the profiles.

Below we follow the same methodology to build profiles with the category-permission-expectancy tuples. Just note that an increase on the number of considered features leads to an increase in the amount of missing data.

In [68]:

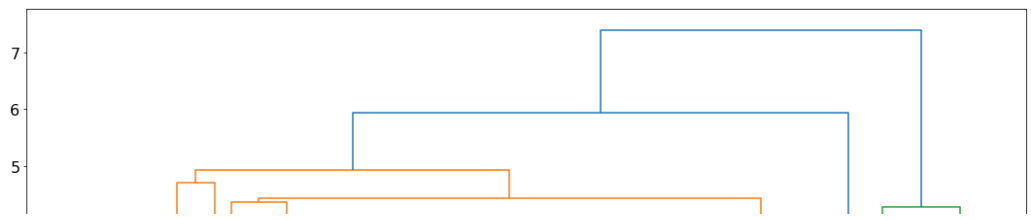
```
data = _get_original_data(db,
                          drop_features=("timestamp",
                                         "requestingApplicationPackageName"),
                          verbose=False)

# Put all games in same category
data['category'] = data['category'].apply(lambda x: x.split('_')[0] if x.startswith('GAME_') else x)

# Clean permission names
data['checkedPermissionGroup'] = data['checkedPermissionGroup'].map(lambda x: x.split('.')[-1])
data = data.rename(columns={'checkedPermissionGroup': 'permission'})
data = data.loc[data['permission'] != 'SENSORS']  # remove SENSORS permissions -- only 1 permission

# Remove requests with wasRequestExpected == UNKNOWN
data = data.loc[data['wasRequestExpected'] != 0]
data['wasRequestExpected'] = data['wasRequestExpected'].replace({1: "UNEXPECTED", 2: "EXPECTED"})

clustering_features = ['category', 'permission', 'wasRequestExpected']
y_feature = 'grantResult'
random_state = 42
tensor_data = _get_tensor_data(data, clustering_features, y_feature, random_state)
```

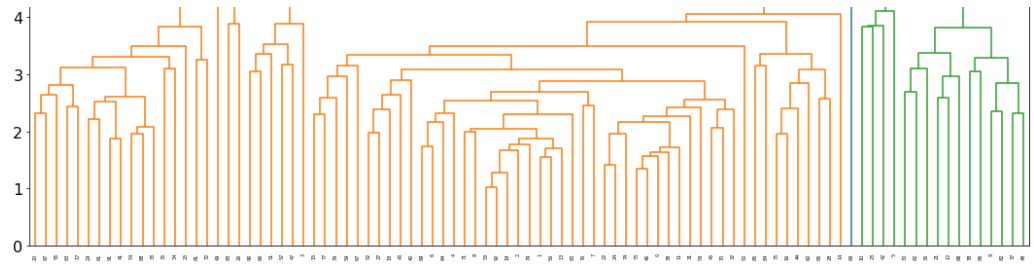
```
Percentage of missing data for dataset with feature ['category', 'permission', 'wasRequestExpected'] is: 84.
03.
```

Dendogram

In [69]:

```
linked = linkage(tensor_data, "complete", optimal_ordering=True)
plt.figure(figsize=(15, 7))
dendrogram(linked, orientation="top", distance_sort="descending", show_leaf_counts=True)
plt.tight_layout()
```



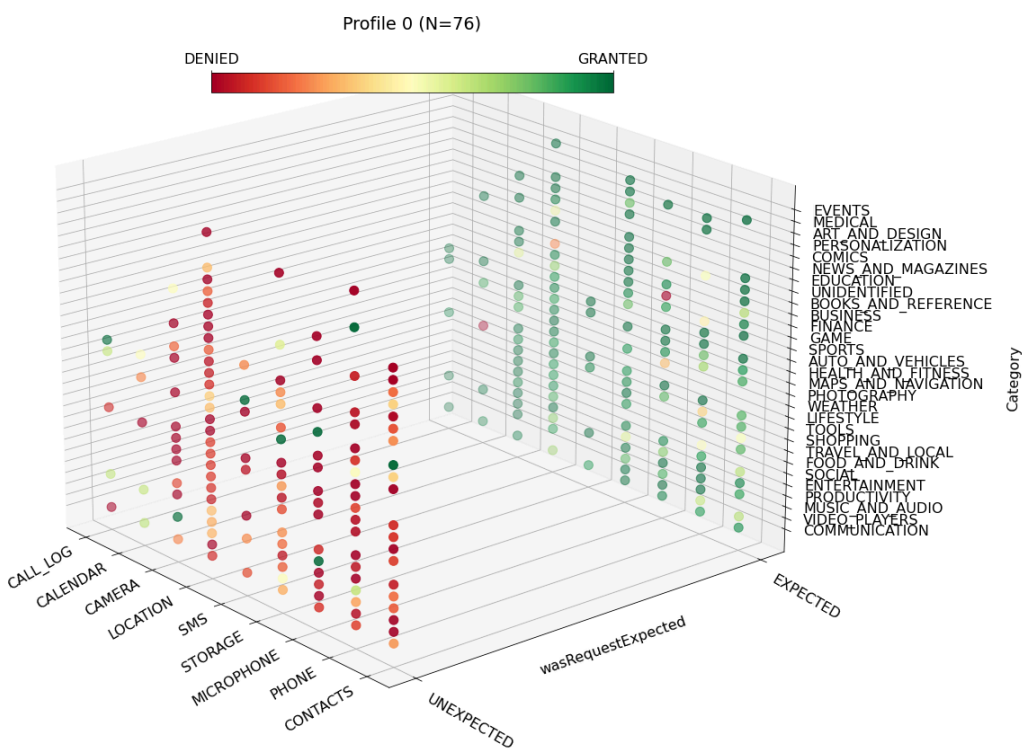


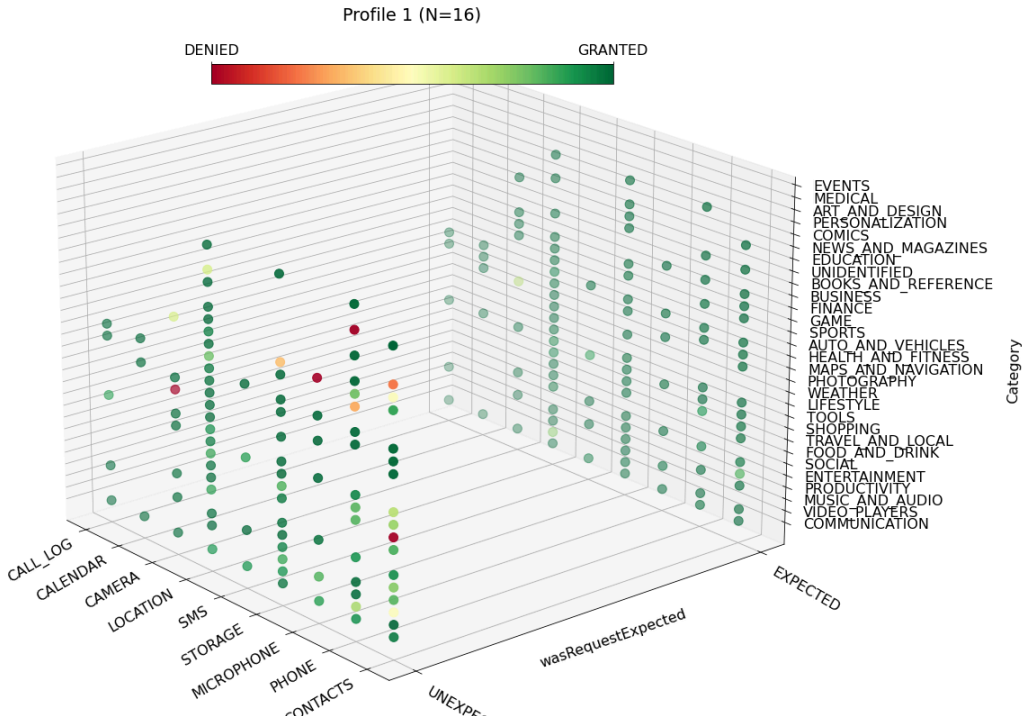
Notice the difference between this dendogram and the previous example. In this case 2 or 3 profiles should result in the best clustering. For an illustrative example we select 3 again.

3 category-permission-expectancy Profiles

In [128…

```
cluster = AgglomerativeClustering(n_clusters=3, affinity="euclidean", linkage="complete")
cluster.fit_predict(tensor_data)
labels_dict = {userID: label for userID, label in
               zip(tensor_data.index, cluster.labels_)}
data["hc_label"] = [labels_dict[userID] for userID in data["userID"]]
# tensor_dataset["hc_label"] = cluster.labels_
permissions = list(data["permission"].unique())
perm_ticks_map = {tick: i for i, tick in enumerate(permissions)}
categories = list(data["category"].unique())
cat_ticks_map = {tick: i for i, tick in enumerate(categories)}

for c in list(set(cluster.labels_)):
    plot_data = data.loc[data["hc_label"] == c, :].groupby(clustering_features).mean()
    plot_data.reset_index(inplace=True)
    fig = plt.figure(figsize=(16, 25))
    ax = fig.add_subplot(projection='3d')
    third_feature_unique_values = list(plot_data[clustering_features[-1]].unique())
    third_feature_ticks_map = {tick: i for i, tick in enumerate(third_feature_unique_values)}
    scatter_plot = ax.scatter(
        [perm_ticks_map[perm] for perm in plot_data["permission"]],
        [third_feature_ticks_map[value] for value in plot_data[clustering_features[-1]]],
        [cat_ticks_map[cat] for cat in plot_data["category"]],
        c=plot_data[y_feature], cmap="RdYlGn",
        s=100)  # , s=100

    ax.set_yticks(range(len(third_feature_unique_values)))
    # ax.set_yticklabels(map(str, third_feature_unique_values))
    ax.set_ylabel(clustering_features[-1])
    ax.set_yticklabels(third_feature_unique_values,
                       rotation=-30,
                       fontdict={'horizontalalignment': 'left',
                                 'verticalalignment': 'top'})

    ax.set_xticks(range(len(permissions)))
    ax.set_xticklabels(permissions, rotation=30,
                       fontdict={
                           'horizontalalignment': 'right',
                           'verticalalignment': 'center_baseline'})
    ax.set_zlabel('Category', labelpad=120)
    ax.set_zticks(range(len(categories)))
    ax.set_zticklabels(categories,
                       fontdict={'horizontalalignment': 'left',
                                 'verticalalignment': 'center_baseline'})
    ax.view_init(20, 140)

    nr_users_in_c = len(data.loc[data["hc_label"] == c, "userID"].unique())
    ax.set_title(f"Profile {c} (N={nr_users_in_c})", y=1.0)

    # cbar = fig.colorbar(cm.ScalarMappable(cmap='RdYlGn'), ax=ax, ticks=[-1, 0], shrink=0.2, pad=.3)
    cbar = fig.colorbar(scatter_plot, ticks=[-1, 0], pad=-0.1, shrink=0.5, location='top')
    cbar.ax.set_xticklabels(['DENIED', 'GRANTED'])
    plt.tight_layout()
```




Profile 1 (N=16)
DENIED
GRANTED
CALL_LOG
CALENDAR
CAMERA
LOCATION
SMS
STORAGE
MICROPHONE
PHONE
CONTACTS
UNEXPE
wasRequestExpected
EXPECTED
EVENTS
MEDICAL
ART_AND_DESIGN
PERSONALIZATION
COMICS
NEWS_AND_MAGAZINES
EDUCATION
UNIDENTIFIED
BOOKS_AND_REFERENCE
BUSINESS
FINANCE
GAME
SPORTS
AUTO_AND_VEHICLES
HEALTH_AND_FITNESS
MAPS_AND_NAVIGATION
PHOTOGRAPHY
WEATHER
LIFESTYLE
TOOLS
SHOPPING
TRAVEL_AND_LOCAL
FOOD_AND_DRINK
SOCIAL
ENTERTAINMENT
PRODUCTIVITY
MUSIC_AND_AUDIO
VIDEO_PLAYERS
COMMUNICATION
Category